

Inteligencia Artificial y Mini-Robots

Taller 3

Presentado por:

Cristian Camilo García Gamboa - crigarciaga@unal.edu.co

Mateo Becerra Porras - mabecerrap@unal.edu.co

Santiago Ospina Contreras - saospinaco@unal.edu.co

Profesor:

José J. Martínez P.

jmcursosingenieria111@gmail.com

Fecha de entrega:

15 de Septiembre de 2026



Universidad Nacional de Colombia - Sede Bogotá
Facultad de Ingeniería
2026 II

Desarrolle tres ejercicios de los 6 propuestos, utilice todas las librerías y herramientas disponibles.

1. Maximizar la función $f(x) = x \sin(10 \pi x) + 1$, con $x \in [0,1]$.

El programa implementa un algoritmo genético para encontrar el valor de x que maximiza la función $f(x) = x \sin(10 \pi x) + 1$, con $x \in [0,1]$.

Para ello, cada posible solución se representa mediante un cromosoma binario de 8 bits, el cual se decodifica a un valor real de x . Inicialmente se genera una población aleatoria de individuos, se evalúa su aptitud mediante la función objetivo y se seleccionan los mejores utilizando el método de ruleta. Posteriormente, los individuos seleccionados se combinan mediante cruce y se modifican aleatoriamente mediante mutación para generar nuevas soluciones. Además, se utiliza elitismo para conservar siempre el mejor individuo encontrado, repitiendo este proceso durante 100 generaciones hasta aproximarse a la solución óptima.

```
Python
import random
import math

# Parámetros del algoritmo genético

N_BITS = 8                # bits del cromosoma
TAM_POBLACION = 20        # número de individuos
N_GENERACIONES = 100      # número de generaciones
PROB_CRUCE = 0.8           # probabilidad de cruce
PROB_MUTACION = 0.02      # probabilidad de mutación por bit
X_MIN, X_MAX = 0.0, 1.0   # rango de la variable x

VALOR_MAX_CROMOSOMA = 2 ** N_BITS - 1  # 255 para 8 bits

# Funciones

def decodificar(cromosoma):
    """Convierte una lista de bits (ej. [1,0,1,1,0,0,1,0]) en un valor real x en [X_MIN, X_MAX]."""
    entero = int("".join(map(str, cromosoma)), 2)
    x = X_MIN + (entero / VALOR_MAX_CROMOSOMA) * (X_MAX - X_MIN)
    return x
```

```

def fitness(cromosoma):
    """Evalúa la función objetivo f(x) para un cromosoma dado."""
    x = decodificar(cromosoma)
    return x * math.sin(10 * math.pi * x) + 1

def crear_individuo():
    """Genera un cromosoma aleatorio de N_BITS bits."""
    return [random.randint(0, 1) for _ in range(N_BITS)]

def crear_poblacion(tam):
    return [crear_individuo() for _ in range(tam)]

def seleccion_ruleta(poblacion, fitnesses):
    """Selección proporcional al fitness (ruleta)."""
    total = sum(fitnesses)
    if total == 0:
        return random.choice(poblacion)
    pick = random.uniform(0, total)
    acumulado = 0
    for individuo, fit in zip(poblacion, fitnesses):
        acumulado += fit
        if acumulado >= pick:
            return individuo
    return poblacion[-1]

def cruce(padre1, padre2):
    """Cruce de un punto."""
    if random.random() < PROB_CRUCE:
        punto = random.randint(1, N_BITS - 1)
        hijo1 = padre1[:punto] + padre2[punto:]
        hijo2 = padre2[:punto] + padre1[punto:]
        return hijo1, hijo2
    return padre1[:], padre2[:]

def mutacion(cromosoma):
    """Mutación bit a bit según PROB_MUTACION."""
    return [bit if random.random() > PROB_MUTACION else 1 - bit for bit in cromosoma]

# Algoritmo genético principal

```

```

def algoritmo_genetico():
    poblacion = crear_poblacion(TAM_POBLACION)
    mejor_individuo = None
    mejor_fitness = -float("inf")
    historial = []

    for generacion in range(N_GENERACIONES):
        fitnesses = [fitness(ind) for ind in poblacion]

        # Guardar el mejor de la generación actual
        idx_mejor = fitnesses.index(max(fitnesses))
        if fitnesses[idx_mejor] > mejor_fitness:
            mejor_fitness = fitnesses[idx_mejor]
            mejor_individuo = poblacion[idx_mejor][:]

        historial.append(mejor_fitness)

        # Elitismo: el mejor individuo pasa directo a la siguiente generación
        nueva_poblacion = [mejor_individuo[:]]

        # Generar el resto de la nueva población
        while len(nueva_poblacion) < TAM_POBLACION:
            padre1 = seleccion_ruleta(poblacion, fitnesses)
            padre2 = seleccion_ruleta(poblacion, fitnesses)
            hijo1, hijo2 = cruce(padre1, padre2)
            nueva_poblacion.append(mutacion(hijo1))
            if len(nueva_poblacion) < TAM_POBLACION:
                nueva_poblacion.append(mutacion(hijo2))

        poblacion = nueva_poblacion

        x_mejor = decodificar(mejor_individuo)
        print(f"Generación {generacion + 1:3d} | "
              f"x = {x_mejor:.6f} | f(x) = {mejor_fitness:.6f}")

    return mejor_individuo, mejor_fitness, historial

```

Ejecución

```

if __name__ == "__main__":
    random.seed(52) # Semilla

    mejor_cromosoma, mejor_valor, historial = algoritmo_genetico()
    x_final = decodificar(mejor_cromosoma)

    print("\n==== RESULTADO FINAL =====")
    print(f"Mejor cromosoma : {mejor_cromosoma}")
    print(f"x óptimo       : {x_final:.6f}")
    print(f"f(x) máximo      : {mejor_valor:.6f}")

```

Durante la ejecución, el algoritmo muestra en cada generación el mejor valor de x encontrado y su correspondiente valor de $f(x)$, permitiendo observar cómo la población evoluciona y mejora progresivamente. Gracias a los operadores genéticos y al elitismo, las soluciones de baja calidad son reemplazadas gradualmente por otras más aptas, haciendo que el valor de la función aumente generación tras generación. Al finalizar, el programa presenta el mejor cromosoma obtenido, el valor óptimo de x asociado y el valor máximo de la función encontrado, que corresponde a la mejor aproximación al máximo global lograda por el algoritmo genético.

Mejor cromosoma: [1, 1, 0, 1, 1, 0, 0, 1]

x óptimo: 0.850980

f(x) máximo: 1.850577

5. Genere aleatoriamente una población de 50 palabras, que se escuche por el parlante del computador. Tomando como función de aptitud una palabra suya, usando AGs, con base en las palabras generadas aleatoriamente llegue a la palabra que usó como función de aptitud.

El programa parte de una población inicial de 50 palabras generadas completamente al azar, las cuales tienen la misma longitud que la palabra objetivo definida como función de aptitud, en este caso "GENETICA". En cada generación, se evalúa qué tan parecida es cada palabra a la palabra objetivo contando la cantidad de letras correctas que aparecen en la posición correcta. A medida que avanzan las generaciones, los individuos con mayor cantidad de coincidencias tienen más probabilidades de ser seleccionados para reproducirse mediante el operador de cruce, mientras que la mutación introduce cambios aleatorios que permiten explorar nuevas combinaciones de letras. Gracias a este proceso evolutivo, las palabras generadas van pareciéndose cada vez más a la palabra objetivo, pasando de secuencias

completamente aleatorias a palabras con varias letras correctas hasta alcanzar finalmente la coincidencia exacta. Adicionalmente, el programa reproduce por el parlante las palabras generadas, permitiendo escuchar cómo la población evoluciona progresivamente hasta encontrar la solución.

Python

```
import random
import string

try:
    import pyttsx3
    TTS_DISPONIBLE = True
except ImportError:
    TTS_DISPONIBLE = False

# Configuración del ejercicio

PALABRA_OBJETIVO = "GENETICA"          # <-- Función de aptitud
ALFABETO = string.ascii_uppercase + " " # letras A-Z + espacio

TAM_POBLACION = 50
PROB_CRUCE = 0.8
PROB_MUTACION = 0.05
N_GENERACIONES = 300

HABLAR = True                          # False desactivar todo el audio
HABLAR_POBLACION_INICIAL = True        # decir las 50 palabras aleatorias iniciales
HABLAR_CADA_GENERACION = True           # decir el mejor individuo de cada generación

# Texto a voz

_motor = None
if TTS_DISPONIBLE and HABLAR:
    try:
        _motor = pyttsx3.init()
        _motor.setProperty("rate", 180)
    except Exception as e:
        print(f"[Aviso] No se pudo inicializar el motor de voz: {e}")
        _motor = None
elif HABLAR and not TTS_DISPONIBLE:
    print("[Aviso] pyttsx3 no está instalado. Instálalo con: pip install pyttsx3")

def decir(texto):
    """Reproduce 'texto' por el parlante si el motor de voz está disponible."""
    if _motor is not None:
        try:
            _motor.say(texto)
```

```

        _motor.runAndWait()
    except Exception as e:
        print(f"[Aviso] Error al reproducir audio: {e}")

# Funciones del algoritmo genético
def crear_individuo(longitud):
    return "".join(random.choice(ALFABETO) for _ in range(longitud))

def crear_poblacion(tam, longitud):
    return [crear_individuo(longitud) for _ in range(tam)]

def fitness(individuo, objetivo):
    """Aptitud = número de letras que coinciden en la misma posición."""
    return sum(1 for a, b in zip(individuo, objetivo) if a == b)

def seleccion_torneo(poblacion, fitnesses, k=3):
    """Selecciona al mejor de k individuos elegidos al azar (torneo)."""
    participantes = random.sample(list(zip(poblacion, fitnesses)), k)
    participantes.sort(key=lambda par: par[1], reverse=True)
    return participantes[0][0]

def cruce(padre1, padre2):
    """Cruce de un punto."""
    if random.random() < PROB_CRUCE and len(padre1) > 1:
        punto = random.randint(1, len(padre1) - 1)
        hijo1 = padre1[:punto] + padre2[punto:]
        hijo2 = padre2[:punto] + padre1[punto:]
        return hijo1, hijo2
    return padre1, padre2

def mutacion(individuo):
    """Mutación letra a letra según PROB_MUTACION."""
    letras = list(individuo)
    for i in range(len(letras)):
        if random.random() < PROB_MUTACION:
            letras[i] = random.choice(ALFABETO)
    return "".join(letras)

# Algoritmo genético principal
def algoritmo_genetico():
    longitud = len(PALABRA_OBJETIVO)
    poblacion = crear_poblacion(TAM_POBLACION, longitud)

    print(f"Palabra objetivo (función de aptitud): {PALABRA_OBJETIVO}\n")
    print("Población inicial de 50 palabras aleatorias:")

```

```

for palabra in poblacion:
    print(f"  {palabra}")

if HABLAR and HABLAR_POBLACION_INICIAL:
    print("\nReproduciendo la población inicial por el parlante...")
    for palabra in poblacion:
        decir(palabra)

mejor = None
for generacion in range(1, N_GENERACIONES + 1):
    fitnesses = [fitness(ind, PALABRA_OBJETIVO) for ind in poblacion]

    idx_mejor = fitnesses.index(max(fitnesses))
    mejor = poblacion[idx_mejor]
    mejor_fit = fitnesses[idx_mejor]

    print(f"Generación {generacion:3d} | Mejor: '{mejor}' | Aciertos:
{mejor_fit}/{longitud}")

    if HABLAR and HABLAR_CADA_GENERACION:
        decir(mejor)

    if mejor == PALABRA_OBJETIVO:
        print(f"\n¡Palabra objetivo encontrada en la generación {generacion}!")
        break

# Elitismo: el mejor pasa directo a la siguiente generación
nueva_poblacion = [mejor]
while len(nueva_poblacion) < TAM_POBLACION:
    padre1 = seleccion_torneo(poblacion, fitnesses)
    padre2 = seleccion_torneo(poblacion, fitnesses)
    hijo1, hijo2 = cruce(padre1, padre2)
    nueva_poblacion.append(mutacion(hijo1))
    if len(nueva_poblacion) < TAM_POBLACION:
        nueva_poblacion.append(mutacion(hijo2))

poblacion = nueva_poblacion

print("\n===== RESULTADO FINAL =====")
print(f"Palabra objetivo   : {PALABRA_OBJETIVO}")
print(f"Palabra encontrada  : {mejor}")

if HABLAR:
    decir(f"La palabra encontrada es {mejor}")
return mejor

```



```
if __name__ == "__main__":
    random.seed()
    algoritmo_genetico()
```

Los resultados obtenidos demuestran que el algoritmo genético fue capaz de encontrar exitosamente la palabra objetivo "GENETICA" a partir de una población inicial compuesta únicamente por palabras aleatorias. Durante el proceso evolutivo se observó una mejora progresiva en la aptitud de los individuos: en la generación 15 el mejor candidato era "TENETIRA" con 6 de 8 letras correctas, y para la generación 17 ya se había alcanzado "JENETICA", con 7 de 8 letras coincidentes. A partir de ese momento, la población conservó esta solución cercana al óptimo gracias al mecanismo de elitismo, hasta que una combinación de cruce y mutación permitió encontrar la palabra exacta "GENETICA" en la generación 30, alcanzando una aptitud máxima de 8/8 aciertos.

```
Generación 15 | Mejor: 'TENETIRA' | Aciertos: 6/8
Generación 16 | Mejor: 'TENETIRA' | Aciertos: 6/8
Generación 17 | Mejor: 'JENETICA' | Aciertos: 7/8
Generación 18 | Mejor: 'JENETICA' | Aciertos: 7/8
Generación 19 | Mejor: 'JENETICA' | Aciertos: 7/8
Generación 20 | Mejor: 'JENETICA' | Aciertos: 7/8
Generación 21 | Mejor: 'JENETICA' | Aciertos: 7/8
Generación 22 | Mejor: 'JENETICA' | Aciertos: 7/8
Generación 23 | Mejor: 'JENETICA' | Aciertos: 7/8
Generación 24 | Mejor: 'JENETICA' | Aciertos: 7/8
Generación 25 | Mejor: 'JENETICA' | Aciertos: 7/8
Generación 26 | Mejor: 'JENETICA' | Aciertos: 7/8
Generación 27 | Mejor: 'JENETICA' | Aciertos: 7/8
Generación 28 | Mejor: 'JENETICA' | Aciertos: 7/8
Generación 29 | Mejor: 'JENETICA' | Aciertos: 7/8
Generación 30 | Mejor: 'GENETICA' | Aciertos: 8/8

¡Palabra objetivo encontrada en la generación 30!

===== RESULTADO FINAL =====
Palabra objetivo   : GENETICA
Palabra encontrada : GENETICA
```

6. Tome el algoritmo de la dieta y ahora incluya costos. Ahora encuentre una dieta que trate de satisfacer la dieta pero con un costo mínimo. Este es un ejemplo de AG multi-objetivo con dos funciones objetivo.

- Planteamiento del problema:

El problema de la dieta consiste en encontrar una combinación de alimentos que cumpla con ciertos requerimientos nutricionales (proteínas, carbohidratos, grasas, vitaminas, etc.).

Ahora se añade un segundo objetivo: minimizar el costo total de la dieta. Por lo tanto, tenemos dos funciones objetivo:

1. Función nutricional: minimizar la desviación respecto a los requerimientos nutricionales.
2. Función económica: minimizar el costo total de los alimentos seleccionados.

Este es un caso típico de optimización multi-objetivo, donde se busca un equilibrio entre cumplir la dieta y gastar lo menos posible.

- Representación

- Cromosoma: vector que representa cantidades de cada alimento (puede ser binario o real).
- Gen: cada posición del cromosoma corresponde a un alimento.
- Fenotipo: dieta resultante (lista de alimentos con cantidades).
- Función de aptitud: combina nutrición y costo.

- Funciones objetivo:

Nutrición:

$$f_1(x) = \sum_j |\text{nutriente}_j(x) - \text{requerimiento}_j|$$

Costo:

$$f_2(x) = \sum_i \text{cantidad}_i \cdot \text{precio}_i$$

- Estrategias de optimización:

Existen varias formas de manejar múltiples objetivos en un AG:

- Método de agregación (ponderación): Se combinan las funciones en una sola, donde alpha y beta son pesos que reflejan la importancia relativa de nutrición y costo.

$$F(x) = \alpha f_1(x) + \beta f_2(x)$$

- Frente de Pareto: Se busca un conjunto de soluciones no dominadas (ninguna es mejor en ambos objetivos a la vez). El resultado es un conjunto de dietas óptimas que representan distintos compromisos entre nutrición y costo.
- Flujo del algoritmo

1. Inicialización: generar poblaciones aleatorias de dietas.

2. Evaluación: calcular $f_1(x)$ y $f_2(x)$ para cada cromosoma.

3. Selección: elegir padres según su desempeño multi-objetivo (ruleta, torneo, o ranking de Pareto).

4. Cruce: combinar dietas para generar nuevas soluciones.
5. Mutación: variar cantidades de alimentos para introducir diversidad.
6. Reemplazo: formar la nueva población.
7. Iteración: repetir hasta alcanzar el número de generaciones o convergencia.

Python

```
import random

# Lista de alimentos con nutrientes y costo
alimentos = [
    {"nombre": "Arroz", "proteina": 2, "carbohidratos": 28, "grasas": 0.3, "costo": 0.5},
    {"nombre": "Pollo", "proteina": 25, "carbohidratos": 0, "grasas": 3, "costo": 2.0},
    {"nombre": "Leche", "proteina": 8, "carbohidratos": 12, "grasas": 5, "costo": 1.2},
    {"nombre": "Manzana", "proteina": 0.3, "carbohidratos": 14, "grasas": 0.2, "costo":
0.8},
    {"nombre": "Lentejas", "proteina": 9, "carbohidratos": 20, "grasas": 0.4, "costo":
1.0},
]

# Requerimientos nutricionales diarios (simplificados)
req_proteina = 50
req_carbohidratos = 130
req_grasas = 30

# -----
# Funciones auxiliares
# -----
def generar_poblacion(K, n_alimentos):
    """Genera población inicial con cantidades aleatorias de alimentos"""
    return [[random.randint(0, 5) for _ in range(n_alimentos)] for _ in range(K)]
def evaluar(cromosoma):
    """Evalúa un cromosoma en dos objetivos: nutrición y costo"""
    proteina = sum(cromosoma[i] * alimentos[i]["proteina"] for i in
range(len(alimentos)))
    carbohidratos = sum(cromosoma[i] * alimentos[i]["carbohidratos"] for i in
range(len(alimentos)))
    grasas = sum(cromosoma[i] * alimentos[i]["grasas"] for i in range(len(alimentos)))
    costo = sum(cromosoma[i] * alimentos[i]["costo"] for i in range(len(alimentos)))

    # Desviación nutricional (cuanto más cerca a los requerimientos, mejor)
    desviacion = abs(req_proteina - proteina) + abs(req_carbohidratos - carbohidratos) +
abs(req_grasas - grasas)

    return desviacion, costo
```

```

def seleccion(poblacion):
    """Selección por torneo"""
    padres = []
    for _ in range(len(poblacion)):
        a, b = random.sample(poblacion, 2)
        if evaluar(a) < evaluar(b):
            padres.append(a)
        else:
            padres.append(b)
    return padres

def cruce(padres):
    """Cruce de un punto"""
    hijos = []
    for i in range(0, len(padres), 2):
        p1, p2 = padres[i], padres[(i+1) % len(padres)]
        punto = random.randint(1, len(p1)-1)
        h1 = p1[:punto] + p2[punto:]
        h2 = p2[:punto] + p1[punto:]
        hijos.extend([h1, h2])
    return hijos

def mutacion(poblacion, p_mut=0.1):
    """Mutación aleatoria"""
    for crom in poblacion:
        for i in range(len(crom)):
            if random.random() < p_mut:
                crom[i] = max(0, crom[i] + random.choice([-1, 1]))
    return poblacion

# -----
# Algoritmo Genético
# -----

def algoritmo_genetico(K=20, generaciones=50, p_mut=0.1):
    poblacion = generar_poblacion(K, len(alimentos))

    for _ in range(generaciones):
        padres = seleccion(poblacion)
        hijos = cruce(padres)
        poblacion = mutacion(hijos, p_mut)

    # Evaluar resultados finales
    evaluaciones = [evaluar(crom) for crom in poblacion]
    # Ordenar por nutrición y costo (multiobjetivo simple)
    poblacion_ordenada = sorted(zip(poblacion, evaluaciones), key=lambda x: (x[1][0],
x[1][1]))

```

```

    return poblacion_ordenada[:5] # Devuelve las 5 mejores soluciones

# -----
# Ejemplo de uso
# -----
if __name__ == "__main__":
    soluciones = algoritmo_genetico()
    for sol, (desv, costo) in soluciones:
        print("Dieta:", sol)
        print("Desviación nutricional:", desv, "Costo:", costo)
        print("-"*40)

```

Este problema ejemplifica cómo los **Algoritmos Genéticos multi-objetivo** permiten encontrar soluciones que equilibran distintos criterios.

En este caso, se logra una dieta que satisface las necesidades nutricionales y minimiza el costo, ofreciendo al usuario varias alternativas óptimas en lugar de una única solución.

Resultado:

```

Dieta: [3, 1, 4, 0, 0]
Desviación nutricional: 21.1 Costo: 8.3
-----

```